



Linear Algebra in video Games

Vertex Shading and Fragment Shading

Osama Islam Derbalah
Hashem Hamed Elwelily
Abdelrhman Ahmed Shawky

Submitted to: Dr. Waleed Abdelmeguid Mohamed

Course: Linear Algebra and Vector Geometry - MATH
201

May 10, 2026

Contents

1	Introduction and Background	2
2	The Video Game Rendering Pipeline	2
2.1	Vertex shading	3
2.2	Rasterization	3
2.3	Fragment Shading	3
3	Linear Algebra Methodology	4
3.1	Homogeneous Coordinates	4
3.2	coordinate transformations	5
4	Vertex Shading	6
4.1	From Model Space to world Space	6
4.2	From World Space to Camera Space	7
4.3	clipping	8
4.4	From NDC to Screen Space (3D to 2D scaling)	9
5	Fragment Shading	10
6	Conclusion	11

Our project examines the video game rendering pipeline from a linear algebra perspective, focusing on vertex and fragment shading. It demonstrates how matrices and vector transformations are used in the vertex shader to convert 3D models into 2D screen coordinates through model, view, and projection transformations. It also explores how linear interpolation of vertex attributes in the fragment shader produces smooth color transitions and realistic surface effects by calculating the cross product and dot product of surface normal vectors and light vectors. Finally, we will demonstrate how these principles work in practice with a functional Python code that takes a group of 3D shapes through the entire rendering process

1 Introduction and Background

Over the last 50 years, video games evolved from simple pixel based arcade games for children into massive cinematic worlds with realistic graphics, complete physics, and intricate plots and characters. Early games like Pong were very limited by hardware, while modern titles such as Cyberpunk 2077 are able to utilize modern graphics card which have monster-like processing power to achieve realistic and beautiful graphic, but what exactly are video games? And How are Graphics cards that only operate in 0s and 1s able to produce them?

video games are interactive pieces of media that are shown to the viewer as a continues feed of video footage, usually made up of around 60 frames per second, every frame is made up of millions of vertices that make up millions of triangles which are the building blocks of any video game scene, these vertices and triangles are represented by their coordinates in the game world which are stored as homogeneous matrices.

as the player goes through the game world, he moves, the characters around him move, the environment moves, and the camera moves, which means that the scene or image that needs to be rendered for every frame is different from the one before it, and an average modern gaming experience usually runs at 60 fps per second, meaning that the Graphics Card in the computer needs to render 60 different images per second, and to achieve that, the Graphics Card applies different matrix transformations and Matrix operations on The matrices that represent the coordinates of each shape, object, and character in the scene so they appear ordered and consistent while moving as you see them while you play the game.

The Graphics card needs to do this 60 times per second, which is why modern gaming Graphics Cards, have evolved to be mathematical processing monsters, capable of handling 3,352 trillion mathematical operations per second.

2 The Video Game Rendering Pipeline

The video feed that appears to you on your monitor is nothing but a series of images strung together to give the illusion of motion, and for each image to look like how you see it on the monitor, it needs to go through a long series of processes in the Graphics Processing unit, these series of Processes go as follows:

2.1 Vertex shading

the process of taking all the objects geometries and meshes as a 3d space and use the field of view of the camera to calculate where each object will be in a 2d window called a view screen, that process is achieved through 4 different space transformations:

1. model space: the original space that stores the object on its own
2. world space: how all the different objects are arranged and mixed together in a specific scene
3. camera space: how the POV camera of the player perceives the scene
4. view screen: the 2D space that gets projected onto your monitor

these transitions are achieved through matrix transformations that are applied to the Homogeneous coordinates matrices of each object in the scene

2.2 Rasterization

using the 3 vertices of a single triangle to figure out which specific pixels on your display are covered by that triangle, and then these pixels are colored using the specific shades of color assigned to that triangle.

in this process we turn triangles into fragments, which are groups of pixels which come from the same triangle and share the same texture or color.

and this step also handles the problem of overlapping triangles or the visibility problem, which is solved by using a depth buffer or a Z buffer that assigns a z value to each vertex and triangle, the frame buffer picks the triangle with the highest Z value to view on the screen while ignoring all other triangles with lower Z values.

2.3 Fragment Shading

a scene that has only vertices representing object geometrics with their respective color values applied to each pixel looks ugly and unrealistic, one very important step needs to be added in order to make the scene look good, which is accounting for the lighting, shading, and reflections produced by different light sources and objects in the scene.

The lighting values are calculated through the normal vectors of different surfaces which are also represented by vector matrices, if a surface is pointed directly towards a light source such as the sun, it's shaded brighter, while it's shaded darker if the normal vector is perpendicular to the light vector or it's facing away from it.

The GPU calculates the lighting value for each triangle while accounting for the different lighting sources in the scene, following that by applying special texture properties into special surfaces, like shininess for metallic surfaces and reflectivity for mirrors and glass, all these additions make the scene look way more realistic and beautiful.

These 3 steps are the main stages of the modern Video game rendering pipeline, although some other minor stages are added in more graphically intense games, but in our project we will focus mainly on Vertex shading and Fragment Shading, because both of them rely

heavily on Linear Algebra concepts in their processes, while rasterization relies on Calculus and other mathematical tools.

3 Linear Algebra Methodology

3.1 Homogeneous Coordinates

The Main principle that Vertex Shading is built on is Linear transformation, taking in a 2-vector and producing another 2×2 vector by a simple matrix multiplication, By this simple formula we can achieve a variety of useful transformations, depending on what we put in the entries of the matrix.

We can use it to scale a vector:

$$\begin{bmatrix} s_x & 0 \\ 0 & s_y \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} s_x x \\ s_y y \end{bmatrix}.$$

To rotate it:

$$\begin{bmatrix} \cos \frac{-\pi}{6} & -\sin \frac{-\pi}{6} \\ \sin \frac{-\pi}{6} & \cos \frac{-\pi}{6} \end{bmatrix} = \begin{bmatrix} 0.866 & 0.5 \\ -0.5 & 0.866 \end{bmatrix}.$$

or to reflect it:

$$\text{reflect-y} = \begin{bmatrix} -1 & 0 \\ 0 & 1 \end{bmatrix}, \quad \text{reflect-x} = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}.$$

It is common for graphics programs to apply more than one transformation to an object, and luckily, we can represent the effects of transforming a vector by two matrices in sequence using a single matrix of the same size, which we can compute by multiplying the two matrices: $M = RS$

$$\begin{bmatrix} 0.707 & -0.707 \\ 0.707 & 0.707 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 0.5 \end{bmatrix} = \begin{bmatrix} 0.707 & -0.353 \\ 0.707 & 0.353 \end{bmatrix}.$$

Note that the order of multiplication is critical in this case, so It is very important to remember that these transforms are applied from the right side first. So the matrix $M = RS$ first applies S and then R.

3D transformations work mostly in the same way as 2D transformations, as you simply have to use a 3×3 matrix for the transformation instead of 2×2

$$\text{scale}(s_x, s_y, s_z) = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & s_z \end{bmatrix}.$$

$$\text{rotate-z}(\phi) = \begin{bmatrix} \cos \phi & -\sin \phi & 0 \\ \sin \phi & \cos \phi & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

However, We cannot use such transforms to move objects, only to scale and rotate them. To move or translate an object by shifting all its points the same amount, we need a transform of the form:

$$\begin{aligned}x' &= x + x_t, \\y' &= y + y_t.\end{aligned}$$

There are multiple ways to achieve this, but the most efficient and simple way to do it is to use affine transformations, we represent the point (x, y, z) by a 4D vector $[x \ y \ z \ 1]^T$, and use 3×3 matrices of the form

$$\begin{bmatrix} m_{11} & m_{12} & m_{13} & x_t \\ m_{21} & m_{22} & m_{23} & y_t \\ m_{31} & m_{32} & m_{33} & z_t \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

The fixed fourth row serves to copy the 1 into the transformed vector, so that all vectors have a 1 in the last place, and the first three rows compute x and y and z as linear combinations of x, y, z, and 1:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} m_{11} & m_{12} & m_{13} & x_t \\ m_{21} & m_{22} & m_{23} & y_t \\ m_{31} & m_{32} & m_{33} & z_t \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} = \begin{bmatrix} m_{11}x & m_{12}y & m_{13}z & x_t \\ m_{21}x & m_{22}y & m_{23}z & y_t \\ m_{31}x & m_{32}y & m_{33}z & z_t \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

That is how we implement a linear transformation followed by a translation in a single matrix, these type of matrices are called Homogeneous coordinates matrices, and they're the cleanest and most straight forward way to apply multiple rigid body transformations.

3.2 coordinate transformations

being able to move the coordinates of each object around using Homogeneous transformations is very useful, but it's only part of the equation for vertex shading, the other part is moving these objects from one coordinate system to another.

initially, the object is stored in its own file or "space" (usually called model space), and the coordinates of every vertex in that object is stored in terms of the coordinate system of that space with the center of the object as its origin point. but in order to build a scene in a video game, we need to mix all these objects together along with the models for the characters to produce a coherent scene, and to do that we move the object from it's original coordinate system to a new coordinate system that contains all the objects in their respective positions in the scene, with the player as its origin point (usually called world space).

Geometrically, a coordinate system, or coordinate frame, consists of an origin and a basis: a set of three vectors that serves as the foundation of the coordinate system, like the i and j and k vectors in our original coordinate system. usually the basis used for representing a coordinate system are Orthonormal bases which are a set of vectors in an inner product space that are both orthogonal (mutually perpendicular, with an inner product of 0) and

normalized (each vector has a unit length of 1). In a frame with origin p and basis u, v, w , the coordinates (u, v, w) describe the point

$$p + u\mathbf{u} + v\mathbf{v} + w\mathbf{w}$$

to transform a point (u, v, w) from its original coordinate system (object space) into a new coordinate system (world space), we use something called frame-to-canonical matrix, which gives us in the new coordinates for the object when multiplied by the Homogeneous coordinates matrix of that object

$$\begin{bmatrix} x_p \\ y_p \\ z_p \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & x_e \\ 0 & 1 & 0 & y_e \\ 0 & 0 & 1 & z_e \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_u & x_v & x_w & 0 \\ y_u & y_v & y_w & 0 \\ z_u & z_v & z_w & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} u_p \\ v_p \\ w_p \\ 1 \end{bmatrix}$$

$$\mathbf{p}_{xyz} = \begin{bmatrix} \mathbf{u} & \mathbf{v} & \mathbf{w} & \mathbf{e} \\ 0 & 0 & 0 & 1 \end{bmatrix} \mathbf{p}_{uvw},$$

4 Vertex Shading

Vertex Shading could be argued to be the most important step of the whole video game graphics rendering pipeline, and it's basically the process of Computing the position of each vertex in the scene that appears to the player, but in order to arrive at that scene gets projected onto the 2D plane that is the monitor the player is looking at, the image has to go through multiple processing stages

4.1 From Model Space to world Space

At the very beginning of the processing timeline, the coordinates for the vertices of the object is stored in a separate file, we call that the object space, and we have to arrange and move around multiple objects and models to construct the required scene, the space that all the objects are arranged in together is called world space. that is the very first step of vertex shading, building our world space.

To move an object from it's original space to the world space, we have to use the frame-to-canonical matrix as we discussed in section 3.2, after we specify the origin point for the scene and its orthonormal basis

$$\begin{bmatrix} x_p \\ y_p \\ z_p \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & x_e \\ 0 & 1 & 0 & y_e \\ 0 & 0 & 1 & z_e \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_u & x_v & x_w & 0 \\ y_u & y_v & y_w & 0 \\ z_u & z_v & z_w & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} u_p \\ v_p \\ w_p \\ 1 \end{bmatrix}$$

$$\mathbf{p}_{xyz} = \begin{bmatrix} \mathbf{u} & \mathbf{v} & \mathbf{w} & \mathbf{e} \\ 0 & 0 & 0 & 1 \end{bmatrix} \mathbf{p}_{uvw},$$

But more often than not, that alone is not enough, we usually have to scale, rotate, and translate the object in order to fit the environment of the scene, we use the process discussed

in section 3.1 to produce the matrix that can apply all our required transformations to the Homogeneous coordinates matrix in one go.

let's say that S is the matrix for the scaling transformation, R is the Matrix for the rotation, and T is the translation matrix, the composite modeling matrix M is $M = T \cdot R \cdot S$. Transforms are applied from the right side first, so the object is scaled and rotated in its local space before being translated to its world position.

4.2 From World Space to Camera Space

Now we have arranged our object and character models in the world of the game, great, but where would that world be perceived from exactly? what's the POV (point of view) that we will see the world from?

That is where our second transformation comes in, the transformation from world space into Camera Space, which is often also called eye space or view space, it's how the player perceives the scene, depending on the location of the "camera" depending on whether it's a first person game (The camera is the player's eyes) or a third person game (the camera is sort of flying in the air and we see the player character from up above).

To transform a scene from world space to camera space, we must apply a view transform. This process reorients the world so that the virtual camera is positioned at the origin (0,0,0) and aligned with the coordinate axes.

Before performing the transformation, the camera's "pose" in world space must be defined. This is typically specified using three parameters:

1. Eye Position (e): The location of the camera in the world
2. Gaze Direction (g): The vector indicating where the camera is pointing
3. Up Vector (t): A reference vector used to establish which way is "up" for the camera

From these, an orthonormal basis (u,v,w) is constructed for the camera:

1. w is a unit vector pointing opposite to the gaze direction
2. u points to the right of the camera and is calculated using the cross product of the up vector and w
3. v points to the top of the camera and is the cross product of w and u

To move objects into camera space, we apply two operations in order, first we Translate all points in the scene so that the camera's world position (e) moves to the origin. This is equivalent to translating by the vector $-e$, then we rotate the world to align the camera's basis vectors (u,v,w) with the world's standard basis vectors (x,y,z), these two steps are concatenated into a single view matrix (or camera matrix).

Once this matrix is constructed, we multiply the world-space coordinates of every vertex in the scene by it, The resulting coordinates are in camera space, here is a quick example of how the camera transformation works:

Calculating the Orthonormal Basis (w, u, v) :

$$e = (3, 0, 4)$$

$$g = (-3, 0, -4)$$

$$t = (0, 1, 0)$$

$$w = \frac{-g}{\| -g \|} = \frac{(3, 0, 4)}{\sqrt{3^2 + 0^2 + 4^2}} = (0.6, 0, 0.8)$$

$$u = \frac{t \times w}{\| t \times w \|} = \frac{(0, 1, 0) \times (0.6, 0, 0.8)}{1} = (0.8, 0, -0.6)$$

$$v = w \times u = (0.6, 0, 0.8) \times (0.8, 0, -0.6) = (0, 1, 0)$$

Defining Translation and Rotation Matrices:

$$T = \begin{bmatrix} 1 & 0 & 0 & -e_x \\ 0 & 1 & 0 & -e_y \\ 0 & 0 & 1 & -e_z \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & -3 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & -4 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$R = \begin{bmatrix} u_x & u_y & u_z & 0 \\ v_x & v_y & v_z & 0 \\ w_x & w_y & w_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 0.8 & 0 & -0.6 & 0 \\ 0 & 1 & 0 & 0 \\ 0.6 & 0 & 0.8 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Calculating final View Matrix $(V = R \cdot T)$:

$$V = R \cdot T = \begin{bmatrix} 0.8 & 0 & -0.6 & -(u \cdot e) \\ 0 & 1 & 0 & -(v \cdot e) \\ 0.6 & 0 & 0.8 & -(w \cdot e) \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 0.8 & 0 & -0.6 & 0 \\ 0 & 1 & 0 & 0 \\ 0.6 & 0 & 0.8 & -5 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

4.3 clipping

before transforming the scene from camera space to viewing space (the 2D image that will be projected onto the monitor) there is a small step that we have to do in order to make the scene look like what we usually see from our eyes, and that is clipping.

First we have to apply a projection matrix to the vertices in camera space. This transformation defines the view volume which is the portion of the 3D world that the camera can actually see. and to make the image realistic we have to mimic human vision by making objects appear smaller as they get farther away, that is why The view volume is a frustum (a truncated pyramid).

Then the geometry is clipped against the boundaries of the view volume. Primitives entirely outside are discarded, those entirely inside are kept, and those intersecting the boundaries are clipped to create new vertices at the intersection points. This ensures that the system does not waste resources processing geometry that is not visible on the screen.

Once clipped, the 4D homogeneous coordinates are converted back into 3D through perspective division (dividing the x,y, and z components by w).

This step places the vertices into a "canonical view volume" which is typically a unit cube ranging from -1 to 1 on each axis (x,y,z), which is called Normalized Device Coordinates (NDC).

4.4 From NDC to Screen Space (3D to 2D scaling)

Now we have the scene that the player is supposed to see from his point of view, but the problem is, that scene is in 3D, and the final image that is supposed to be produced is in 2D (projected onto a 2D plane which is the monitor), so how exactly are we supposed to transform these 3D coordinates into 2D coordinated that can be viewed on the screen?

to map 3D coordinates into 2D pixel positions, we take the NDC coordinates which are typically a unit cube where x,y,z coordinates range from -1 to 1. The target is a rectangle on the screen defined by its width (n_x) and height (n_y) in pixels.

fundamentally, this is a windowing transformation that uses scaling and translation to fit one rectangle into another, To map a coordinate $x_{canonical}$ to x_{screen} , the formula is:

$$x_{screen} = \frac{n_x}{2}x_{canonical} + \frac{n_x - 1}{2}$$

and the same goes for the Y coordinate:

$$y_{screen} = \frac{n_y}{2}y_{canonical} + \frac{n_y - 1}{2}$$

Although the screen is 2D, the z-coordinate is not discarded. It is scaled and biased (typically to a range of 1) so it can be stored in the z-buffer to handle object occlusion and hidden surface removal in the Rasterization and Fragment Shading stages.

These operations are concatenated into a single viewport matrix M_{vp} . This allows the hardware to transform vertices with a single matrix vector multiplication:

$$M_{vp} = \begin{bmatrix} \frac{n_x}{2} & 0 & 0 & \frac{n_x-1}{2} \\ 0 & \frac{n_y}{2} & 0 & \frac{n_y-1}{2} \\ 0 & 0 & \frac{z_{max}-z_{min}}{2} & \frac{z_{max}+z_{min}}{2} \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Finally, we have arrived at the final destination for the geometrics of the objects included in our video game scene.

5 Fragment Shading

Fragment shading (or pixel shading) is the stage of the rendering pipeline where a final color is computed for every discrete piece of a primitive, called a fragment, that overlaps a pixel, and to compute that final colour we have to account for multiple factors such as texture, lighting, shadows, and the original color assigned to that surface.

This stage begins with the geometric data (normals, positions, texture coordinates) that were output by the vertex shader, the graphics hardware uses barycentric coordinates (u, v, w) to perform a weighted average of these attributes across the triangle's face. barycentric coordinates are a coordinate system that specify a point's location within a simplex (usually a triangle) as a weighted average of its vertices. The unique value at any interior point P is calculated as a linear combination of the values at the vertices:

$$f(P) = u \cdot f_0 + v \cdot f_1 + w \cdot f_2$$

That Process that we just performed is called interpolation, and it results in a vector that is slightly shorter than unit length. Shading equations rely on unit vectors to ensure that dot products represent only the cosine of the angle between directions, so the shader performs a process of normalization, deviding the interloped vector by its magnitude:

$$\mathbf{n} = \frac{\mathbf{n}_{interp}}{\|\mathbf{n}_{interp}\|} \text{ where } \|\mathbf{n}\| = \sqrt{n_x^2 + n_y^2 + n_z^2}$$

after we have obtained the normalized interloped barycentric coordinates for the surfaces, we can finally apply lighting on it, the intensity of the lighting on each surface is a product of 2 things, the normal of the surface, which we have, and the light direction.

in most video game scenes, there are multiple sources of lighting illuminating the view, like fire, electric lamps, torches, or simply the sun. to find the light direction between each light source and each pixel in the scene, we use vector subtraction. The light direction l is the difference between the light's world space position and the interpolated fragment position:

$$\mathbf{l} = \mathbf{P}_{light} - \mathbf{P}_{fragment}$$

Now that we have the light direction for each pixel, let's find the light intensity, which is the final value that will be applied to the targeted surface, and it's Calculated using the dot product between the normal n and light direction l . The intensity is proportional to $n \cdot l$, which is the cosine of the angle between them

$$\cos(\theta) = \mathbf{n} \cdot \mathbf{l}$$

$$\cos(\theta) = (0 \cdot 0.6) + (1 \cdot 0.8) + (0 \cdot 0)$$

$$\cos(\theta) = 0 + 0.8 + 0 = 0.8$$

congratulations! you have just achieved the perfect Video Game scene, although all of this work was just for a single image out of 60 images that need to be processed per second! makes you realize why GPUs need all that monstrous computing power.

6 Conclusion

To sum up, the visually stunning and immersive worlds of modern video games are fundamentally built upon the core principles of linear algebra. As we have shown throughout this project, the entire graphics rendering pipeline from the initial placement of 3D models to the final pixel drawn on the screen, is a continuous execution of matrix multiplications and vector operations. Vertex shading utilizes homogeneous coordinate matrices to translate, rotate, scale, and project objects from their isolated model spaces onto a 2D screen. on the other hand, fragment shading brings life into these flat geometries by applying vector mathematics, such as interpolation, normalization, and the dot product, to calculate realistic lighting and shadows based on surface normals and light directions. Ultimately, linear algebra serves as the invisible mathematical engine driving the video game industry, seamlessly translating raw numerical data into the breathtaking digital experiences we enjoy today.

References

- Akenine-Möller, Tomas (2018). *Real-Time Rendering. Fourth Edition*. CRC Press.
- Dunn, Fletcher and Ian Parberry. (2002). *3D Math Primer for Graphics and Game Development*. Wordware Publishing.
- Hughes John F., Andries van Dam (2014). *Computer Graphics: Principles and Practice. Third Edition*. Pearson Education.
- Marschner, Steve and Peter Shirley (2016). *Fundamentals of Computer Graphics. Fourth Edition*. CRC Press.
- Strang, Gilbert (2011). *Linear Algebra and its applications 4th edition*. Pearson Education.